PES University — Department of CSE
Software Engineering — Laboratory

Final Lab Report

Lab 1 — Requirements Engineering & UML Use-Case Modelling • Lab 3 — Component Modelling & Architectural Pattern Selection

Name	Pranaav P
SRN	PES1UG24AM387
Lab 1 scenario	Urban Rooftop Gardening Co-op App (Problem Statement #65)
Lab 3 scenario	Self-Service Coffee Kiosk System

1. Lab 1 — Requirements Engineering & UML Use-Case Modelling

1.1 Problem Overview

A community urban-agriculture app for registering rooftop plots, logging crop yields and harvest schedules, sharing soil-maintenance tips, and trading surplus produce. Actors: Co-op Gardener and Community Leader.

1.2 Requirements Table

ID	Type	Priority	Description	Acceptance Criteria	Rationale
FR-001	Functional	High	Allow a Co-op Gardener to log seasonal crop planting dates for a registered plot, and compute the estimated harvest window and expected yield weight from crop type and planting date.	Pass: harvest window/alert generated within the predicted window for a valid record. Fail: an invalid planting-date sequence (harvest-before-planting) is accepted.	Core value of the app — plan harvests and anticipate yields.
FR-002	Functional	High	Allow a Co-op Gardener to register a rooftop plot (location, area m², sunlight, owner) and assign each a unique Plot ID.	Pass: plot with all mandatory fields saved and retrievable by Plot ID. Fail: saved with a missing field or duplicate Plot ID.	Plots are the anchor entity for crop logs, yields and trades.
FR-003	Functional	Medium	Allow a gardener to post surplus produce (item, quantity, availability date) to a community trade board and let others reserve/claim	Pass: posted listing visible to others; status → 'Reserved' when claimed. Fail: a claimed listing stays available to others.	Reduces food waste; supports produce sharing.

			listings.		
FR-004	Functional	Medium	Allow members to publish and browse soil-maintenance tips, tagged by topic (composting, pH, pests), newest first.	Pass: published tip appears in its tagged feed within one refresh. Fail: a tip is published without a topic tag.	Builds shared community knowledge.
FR-005	Functional	Low	Allow a Community Leader to broadcast announcements to all members and moderate (hide/remove) rule-violating trade-board listings.	Pass: broadcast reaches all active members; moderated listing removed from the public board. Fail: a non-leader can broadcast or moderate.	Governance; keeps the shared boards trustworthy.
NFR-001	Non-Functional (Performance & Security)	High	The trade board updates surplus listings in near real-time (≤ 2 s) across members, over authenticated, encrypted (TLS) connections.	Pass: benchmarking confirms ≤ 2 s median update latency and TLS-only transport under simulated peak load (200 concurrent members). Fail: latency exceeded or any unencrypted update observed.	Freshness prevents double-claims; encryption protects member data.
NFR-002	Non-Functional (Usability & Availability)	Medium	The app is usable outdoors (WCAG 2.1 AA contrast, touch targets ≥ 44 px) and the backend keeps ≥ 99.5% monthly availability in the growing season.	Pass: accessibility audit shows no AA contrast failures and uptime ≥ 99.5%. Fail: AA failures found or uptime < 99.5%.	Used outdoors in bright light; reliability keeps logging/trading dependable.

Table 1 — 5 Functional + 2 Non-Functional requirements.

1.3 UML Use-Case Diagram

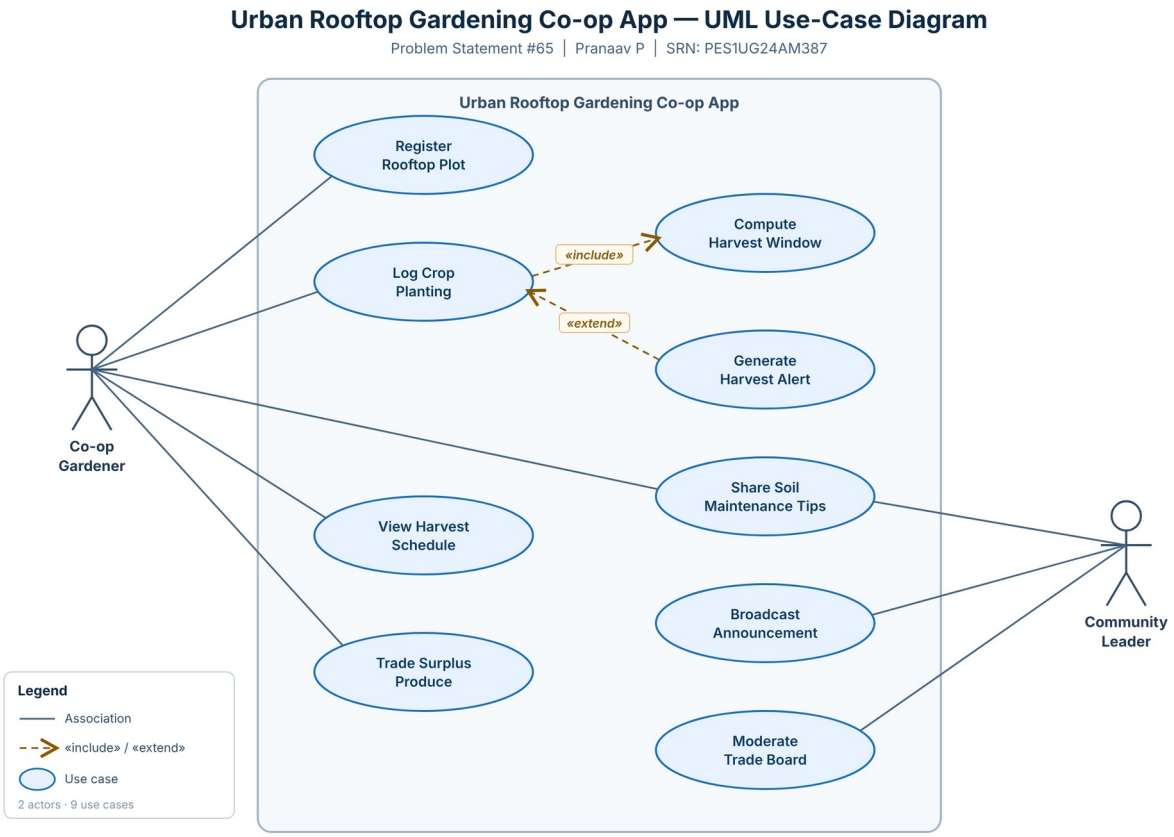


Figure 1 — Use-case diagram. «include»: Log Crop Planting → Compute Harvest Window; «extend»: Generate Harvest Alert → Log Crop Planting.

1.4 Use-Case Flow Specification — Log Crop Planting (UC-02)

Primary actor: Co-op Gardener.

Preconditions: logged in; has at least one registered plot (FR-002); crop catalogue available.

Postconditions: planting record persisted; harvest window + yield computed and stored («include»); harvest alert optionally scheduled («extend»).

Main Success Scenario:

1. Gardener selects a registered plot.
2. System displays the planting form and crop catalogue.
3. Gardener selects crop type and enters the planting date.
4. System validates the planting date (valid, not an impossible sequence).
5. System computes the harvest window and expected yield from growth duration and plot area [«include» Compute Harvest Window].
6. System saves the planting record with the computed window and yield estimate.
7. System confirms and displays the estimated harvest window.
   - **7a.** [«extend» Generate Harvest Alert] If alerts are enabled, schedule a harvest-window notification.

Alternate Flow — A1: Invalid planting date sequence (at step 4):

- **4a.1** System detects an invalid date (earlier than the plot's last harvest, or harvest-before-planting).
- **4a.2** System rejects the entry and shows a validation message.

4a.3 System prompts for a valid date and returns to step 3.

4a.4 If the gardener cancels, no record is saved (the invalid sequence is never accepted).

2. Lab 3 — Component Modelling & Architectural Pattern Selection

2.1 Scenario Overview

A Self-Service Coffee Kiosk for a busy café: customers pick from 3 coffee types (Espresso, Americano, Latte) and 2 sizes (Small, Large), pay by credit card only, and the system prints a receipt. It must drive a touch screen, connect to receipt-printer hardware, and store menu and pricing data.

2.2 Architectural Style Analysis

Style	Structure	Pros (for this scenario)	Cons (for this scenario)	Verdict
Layered	Presentation / Business / Data tiers.	Clear separation of concerns; simple to build & test; fast in-process calls; natural fit for a self-contained device.	Can be rigid; layers can become coupled — but acceptable for a small, fixed feature set.	**Chosen**
Microservices	Small, independently deployable services.	Independent scaling, fault isolation, tech diversity.	Operational complexity, network latency and data-consistency problems — overkill for one kiosk.	**Rejected**
Client-Server	Central server with thin clients.	Centralised control; easy data consistency.	Single point of failure and a remote-server bottleneck; the kiosk must keep working locally at the counter.	**Rejected**

Table 2 — Candidate architectural styles evaluated against the kiosk scenario.

2.3 Chosen Architecture & Components

Layered (3-tier). Five components are identified (Order Manager and Payment Service given; the remaining three added):

- Touchscreen UI (Presentation) — renders the menu, captures touch selections.
- Order Manager (Business, given) — orchestrates the order and coordinates the other components.
- Payment Service (Business, given) — processes the credit-card payment via the EMV reader.
- Menu & Pricing Repository (Data) — stores the 3 coffee types, 2 sizes and prices.
- Receipt Printer Driver (Hardware-abstraction) — formats and drives the thermal printer.

2.4 UML Component Diagram

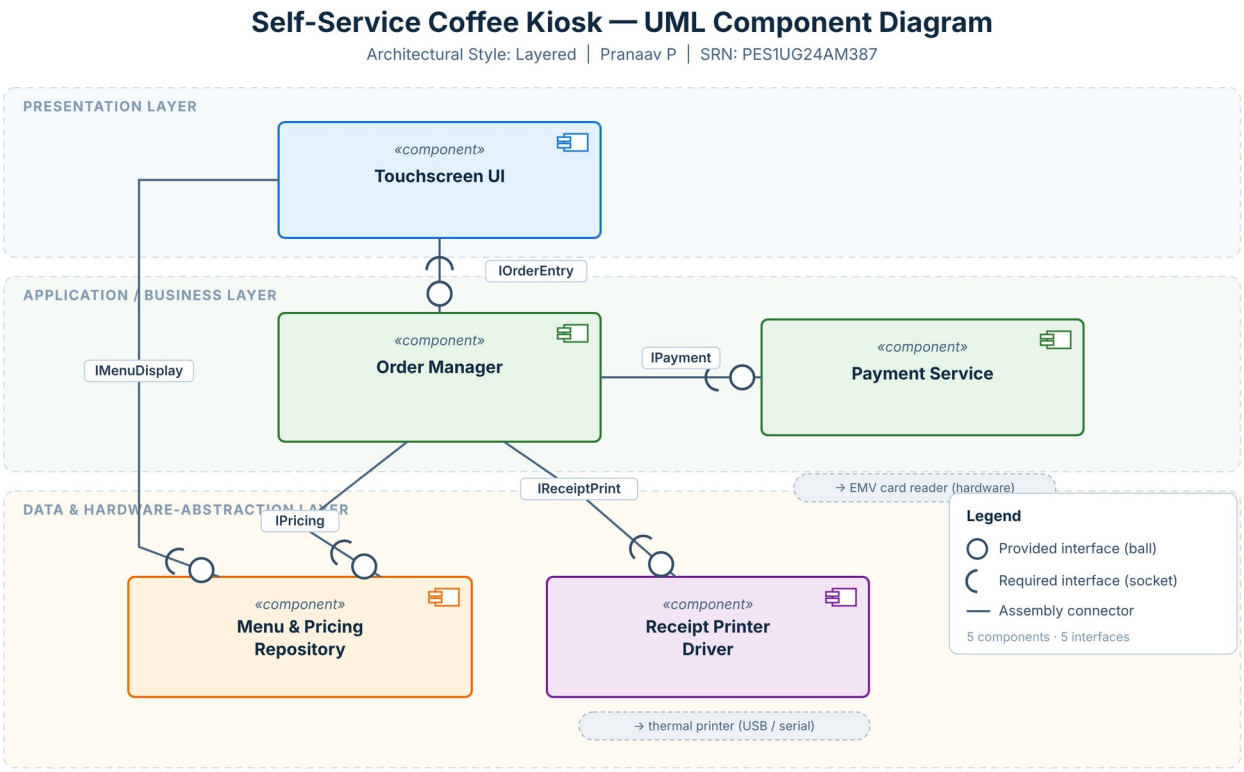


Figure 2 — Component diagram: 5 components, 5 provided/required (ball-and-socket) interfaces (IOrderEntry, IMenuDisplay, IPayment, IPricing, IReceiptPrint).

2.5 Written Justification

We chose a Layered (3-tier) Architecture for the Self-Service Coffee Kiosk System.

Reason 1 — Fit to a simple, fixed workflow: the kiosk is one self-contained device with a small, stable feature set (3 coffee types × 2 sizes → pay → print). Layering maps these onto clear tiers and keeps the fixed workflow easy to build, test and maintain — without the operational and networking overhead a Microservices style would add to a single device.

Reason 2 — No need for distributed scaling: one kiosk serves one customer at a time, so there is no demand for independent scaling or deployment. A layered monolith enforces separation of concerns while avoiding the single-remote-server bottleneck of pure Client-Server and the latency of service-to-service hops.

Security advantage: layering confines the Payment Service to the business tier behind the UI. The Touchscreen UI never handles raw card data or talks to the gateway directly — every payment request funnels through one controlled path (Order Manager → Payment Service → EMV reader). This single boundary shrinks the attack surface, centralises input validation / PCI-style controls, and keeps cardholder data out of the UI layer.

Performance benefit: all layers run in-process on the kiosk, so inter-layer calls are fast local method invocations rather than network round-trips — giving the low, predictable latency a touchscreen needs. Menu and pricing data can be cached in the data tier so the menu renders instantly, even during busy periods.